МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра защиты информации



ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине: «Безопасность систем баз данных»

по теме: «Защита БД студий дубляжа»

Выполнил:

Студент гр. «АБс-322», «АВТФ»

Бутаков Никита Максимович

«___» _____ 2026г

_____

(подпись)

Проверил:

Ассистент кафедры ЗИ

Питько Яна Андреевна

«___» _____ 2026г

_____

(подпись)

Новосибирск 2026

Содержание

Введение 3
1. Применение индексов для оптимизации запросов 4
2. Применение триггеров для автоматизации поддержания целостности данных 9
3. Выводы о применении индексов и триггеров в БД 12
Вывод 14
Приложение А 15
Приложение Б 20

Введение

Цели работы: изучить различные типы индексов в PostgreSQL и их влияние на производительность запросов, а также триггеры для автоматизации поддержания целостности данных в БД.

Тема: «Защита БД студий дубляжа – управление доступом к авторским материалам, контрактам с актёрами, расписаниям записей и защищённым архивам аудиофайлов».

Задачи:

1. Применение индексов для оптимизации запросов;

2. Применение триггеров для автоматизации поддержания целостности данных;

3. Выводы о применении индексов и триггеров в БД.

1. Применение индексов для оптимизации запросов

Для начала создадим отдельную таблицу для тестов индексации app.index_benchmark (Рисунок 1).

	id [PK] bigint	order_code bigint	customer_email text	amount numeric (12,2)	created_at timestamp w	geolocation point
1	1	1000001	client_1@mail.local	6696.35	2026-04-...	(69.84859759825213,-822.87...
2	2	1000002	client_2@mail.local	9755.46	2026-04-...	(770.8409008629146,552.11...
3	3	1000003	client_3@mail.local	5423.70	2026-04-...	(35.661439597660774,996.9...
4	4	1000004	client_4@mail.local	1152.71	2026-04-...	(-634.6443160491403,-649.2...
5	5	1000005	client_5@mail.local	6436.86	2026-04-...	(23.71539246880161,454.77...
6	6	1000006	client_6@mail.local	1025.60	2026-04-...	(253.70586020705582,300.7...
7	7	1000007	client_7@mail.local	3474.87	2026-04-...	(484.8436367662571,-169.97...
8	8	1000008	client_8@mail.local	1813.35	2026-04-...	(-143.37549653033216,935.5...
9	9	1000009	client_9@mail.local	5104.08	2026-04-...	(318.93708960234335,-109.2...

Рисунок 1 – Таблица app.index_benchmark

Далее с помощью EXPLAIN ANALYZE замерим время для выполнения запросов к таблице app.index_benchmark до применения индексации. На рисунке 2 показан анализ запроса B-TREE индекса.

```
                                                        QUERY PLAN
------------------------------------------------------------------------------------------------------------------------
 Gather  (cost=1000.00..11545.10 rows=6 width=205) (actual time=0.995..11.427 rows=6 loops=1)
   Workers Planned: 2
   Workers Launched: 2
   ->  Parallel Seq Scan on index_benchmark  (cost=0.00..10544.50 rows=2 width=205) (actual time=3.165..8.368
 rows=2 loops=3)
         Filter: (customer_email = 'client_12345@mail.local'::text)
         Rows Removed by Filter: 99998
 Planning Time: 0.095 ms
 Execution Time: 11.440 ms
(8 rows)
```

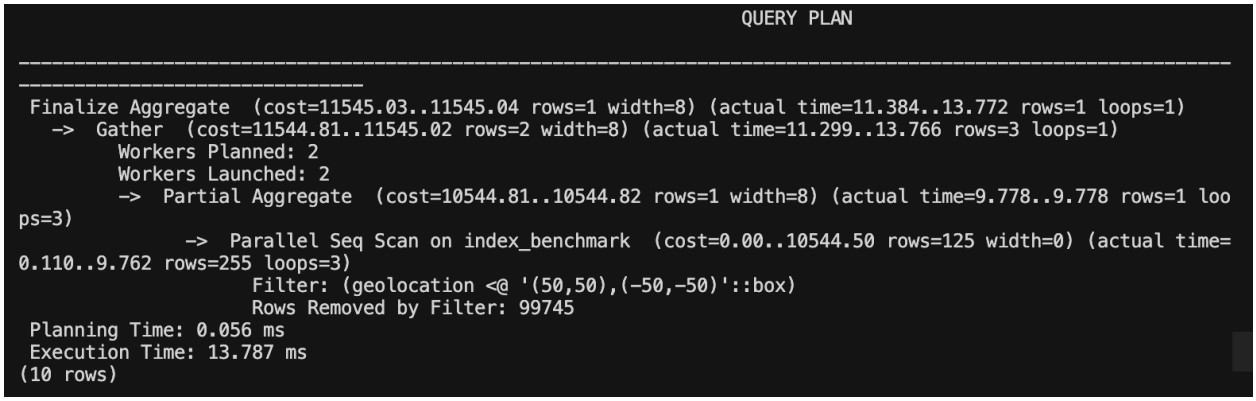
Рисунок 2 – Анализ запроса B-TREE индекса

На рисунке 3 показан анализ запроса Hash индекса.

```
                                                        QUERY PLAN
------------------------------------------------------------------------------------------------------------------------
 Gather  (cost=1000.00..11544.60 rows=1 width=205) (actual time=8.405..9.850 rows=1 loops=1)
   Workers Planned: 2
   Workers Launched: 2
   ->  Parallel Seq Scan on index_benchmark  (cost=0.00..10544.50 rows=1 width=205) (actual time=6.178..7.080
 rows=0 loops=3)
         Filter: (order_code = 1187654)
         Rows Removed by Filter: 100000
 Planning Time: 0.050 ms
 Execution Time: 9.862 ms
(8 rows)
```

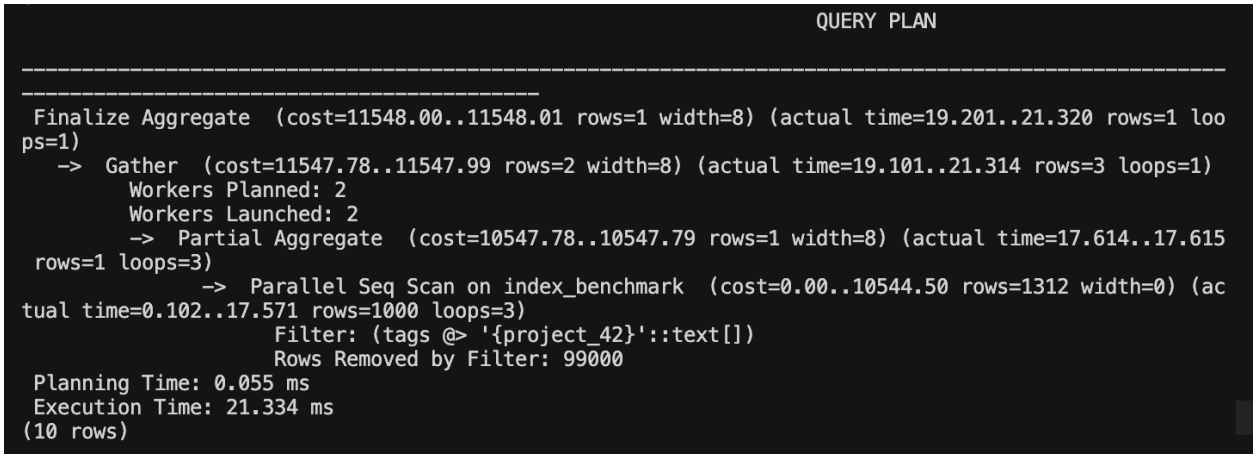
Рисунок 3 – Анализ запроса HASH индекса

На рисунке 4 показан анализ запроса GiST индекса.



```
                                                
```

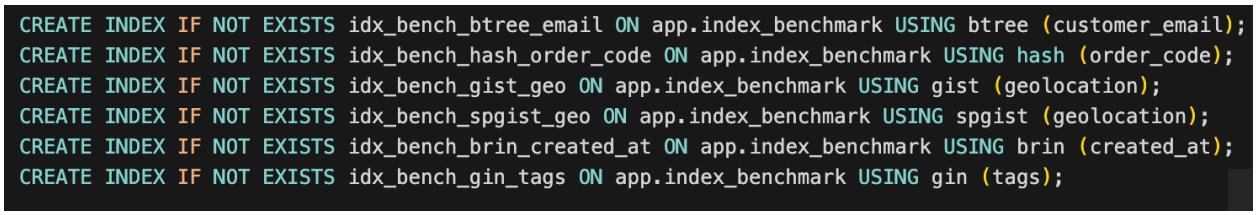
На рисунке 7 показан анализ запроса GIN индекса.



```
                                                                QUERY PLAN
------------------------------------------------------------------------------------------------------------------------
 Finalize Aggregate  (cost=11548.00..11548.01 rows=1 width=8) (actual time=19.201..21.320 rows=1 loops=1)
   ->  Gather  (cost=11547.78..11547.99 rows=2 width=8) (actual time=19.101..21.314 rows=3 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         ->  Partial Aggregate  (cost=10547.78..10547.79 rows=1 width=8) (actual time=17.614..17.615 rows=1 loops=3)
               ->  Parallel Seq Scan on index_benchmark  (cost=0.00..10544.50 rows=1312 width=0) (actual time=0.102..17.571 rows=1000 loops=3)
                     Filter: (tags @> '{project_42}'::text[])
                     Rows Removed by Filter: 99000
 Planning Time: 0.055 ms
 Execution Time: 21.334 ms
(10 rows)
```

Рисунок 7 – Анализ запроса GIN индекса

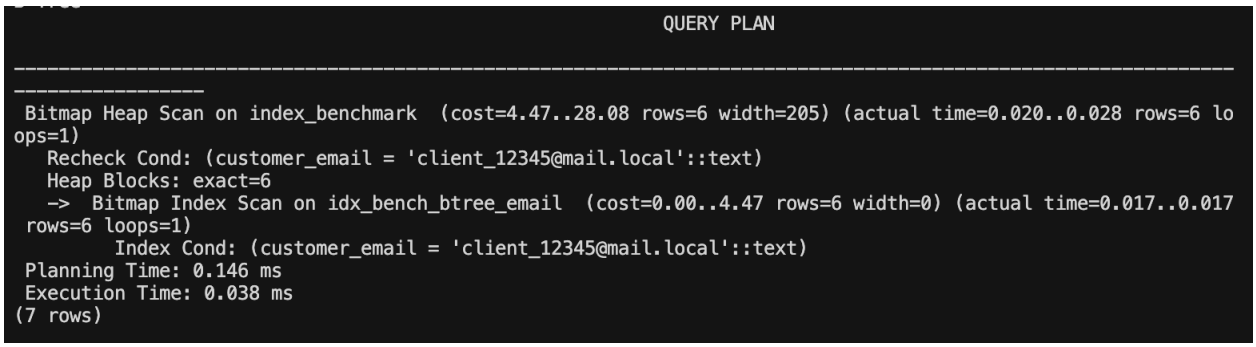
После анализа запросов мы создали B-TREE, Hash, GiST, SP-GiST, BRIN и GIN индексы (Рисунок 8).



```
CREATE INDEX IF NOT EXISTS idx_bench_btree_email ON app.index_benchmark USING btree (customer_email);
CREATE INDEX IF NOT EXISTS idx_bench_hash_order_code ON app.index_benchmark USING hash (order_code);
CREATE INDEX IF NOT EXISTS idx_bench_gist_geo ON app.index_benchmark USING gist (geolocation);
CREATE INDEX IF NOT EXISTS idx_bench_spgist_geo ON app.index_benchmark USING spgist (geolocation);
CREATE INDEX IF NOT EXISTS idx_bench_brin_created_at ON app.index_benchmark USING brin (created_at);
CREATE INDEX IF NOT EXISTS idx_bench_gin_tags ON app.index_benchmark USING gin (tags);
```

Рисунок 8 – Созданные индексы для оптимизации

Теперь проверим анализ запросов после индексации. На рисунке 9 представлен анализ запроса на основе B-TREE индекса.



```
                                                          QUERY PLAN
------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on index_benchmark  (cost=4.47..28.08 rows=6 width=205) (actual time=0.020..0.028 rows=6 loops=1)
   Recheck Cond: (customer_email = 'client_12345@mail.local'::text)
   Heap Blocks: exact=6
   ->  Bitmap Index Scan on idx_bench_btree_email  (cost=0.00..4.47 rows=6 width=0) (actual time=0.017..0.017 rows=6 loops=1)
         Index Cond: (customer_email = 'client_12345@mail.local'::text)
 Planning Time: 0.146 ms
 Execution Time: 0.038 ms
(7 rows)
```

Рисунок 9 – Анализ запроса B-TREE индекса

На рисунке 10 представлен анализ запроса на основе HASH индекса.

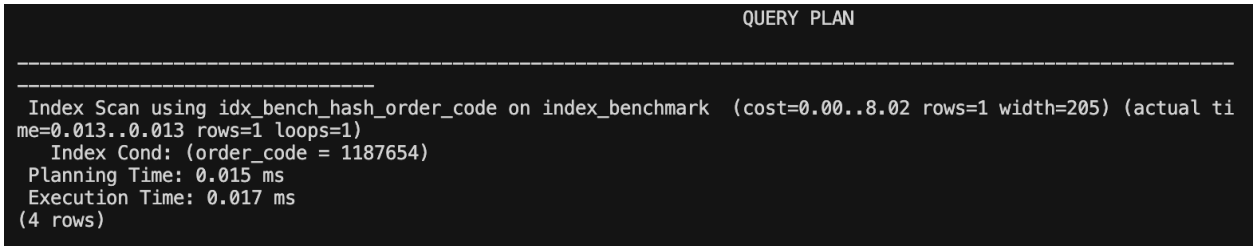


Рисунок 10 – Анализ запроса HASH индекса

На рисунке 11 представлен анализ запроса на основе GiST индекса.

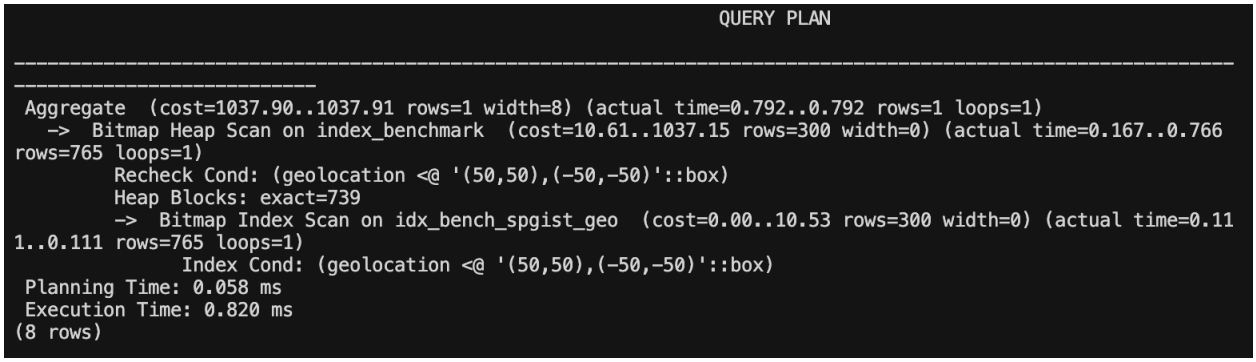


Рисунок 11 – Анализ запроса GiST индекса

На рисунке 12 представлен анализ запроса на основе SP-GiST индекса.

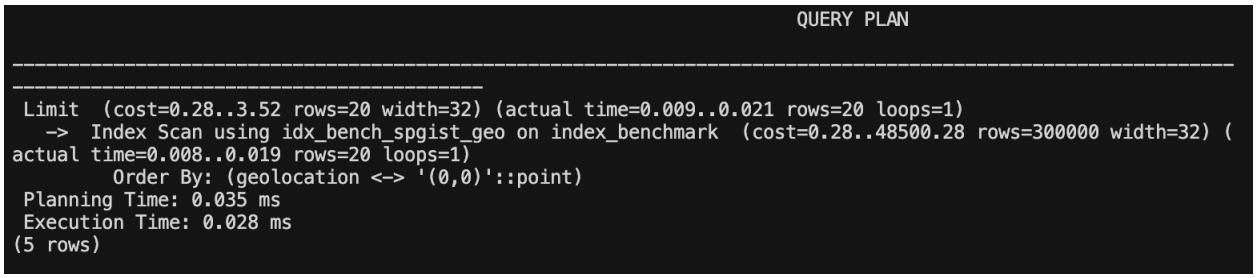


Рисунок 12 – Анализ запроса SP-GiST индекса

На рисунке 13 представлен анализ запроса на основе BRIN индекса.

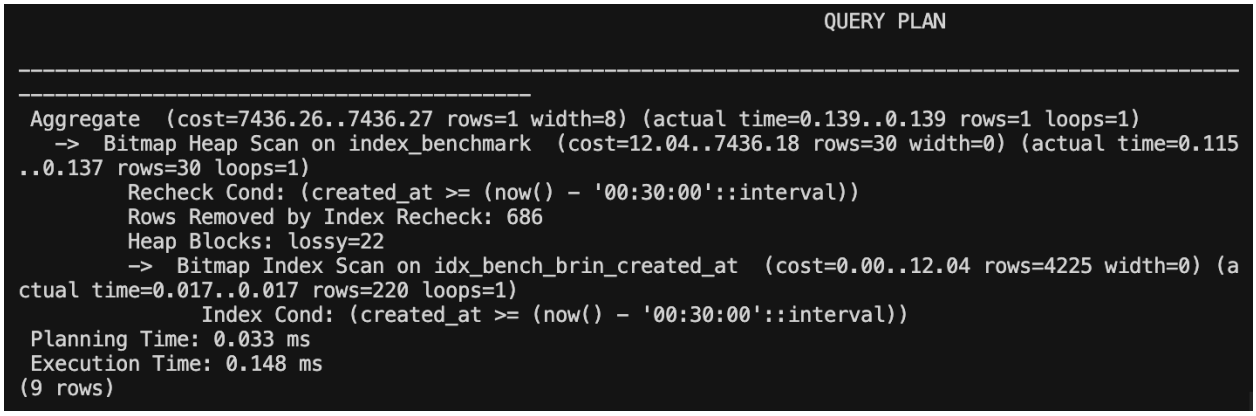


Рисунок 13 – Анализ запроса BRIN индекса

На рисунке 14 представлен анализ запроса на основе GIN индекса.

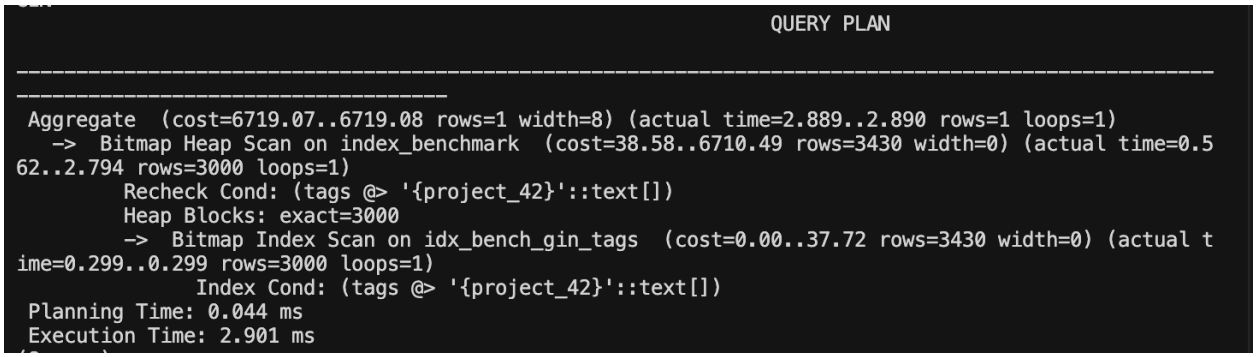


Рисунок 14 – Анализ запроса GIN индекса

В общем можем заметить, что индексы сильно ускоряют запросы. Код применения индексов для оптимизации запросов представлен в приложении А.

2. Применение триггеров для автоматизации поддержания целостности данных

Создадим триггеры для автоматизации поддержания целостности данных. Первый триггер для проверки уникальности значения поля перед вставкой новой записи. Если такое значение уже существует, триггер отменяет операцию вставки и возвращает сообщение об ошибке (Рисунок 15).



Рисунок 15 – Проверка первого триггера

Второй триггер, который обновляет соответствующие поля в связанных таблицах при изменении ключевой записи. Сначала посмотри на исходные данные без изменений. (Рисунок 16).

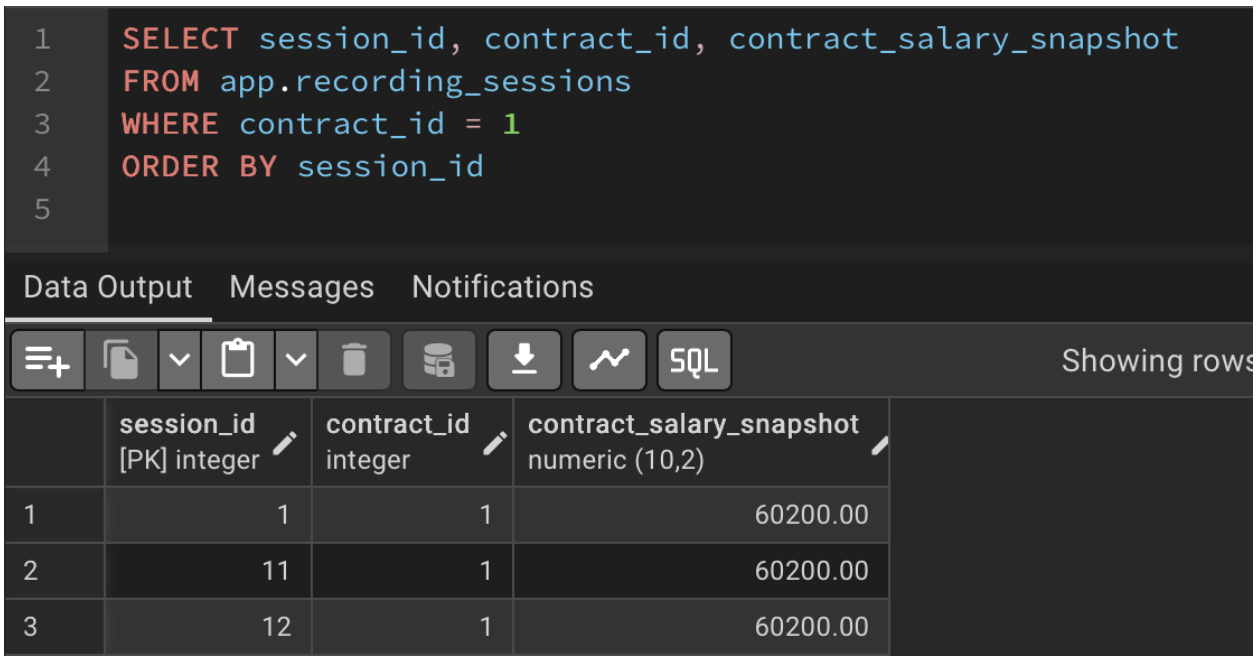


Рисунок 16 – Изначальные данные

После чего обновим зарплату в таблице контрактов на 100 (Рисунок 17).

```
1  UPDATE app.contracts
2  SET salary_amount = salary_amount + 100
3  WHERE contract_id = 1;
```

Data Output | Messages | Notifications

```
UPDATE 1

Query returned successfully in 42 msec.
```

Рисунок 17 – Обновление данных

После обновления суммы заработной платы в таблице контрактов видим, что все связанные данные изменились в таблице сессий (Рисунок 18).

```
1  SELECT session_id, contract_id, contract_salary_snapshot
2  FROM app.recording_sessions
3  WHERE contract_id = 1
4  ORDER BY session_id
```

Data Output | Messages | Notifications

SQL

Showing r

	session_id [PK] integer	contract_id integer	contract_salary_snapshot numeric (10,2)
1	1	1	60300.00
2	11	1	60300.00
3	12	1	60300.00

Рисунок 18 – Проверка второго триггера

Третий триггер, который отслеживает аномальные или подозрительные операции с данными (например, большое количество удалений за короткий промежуток времени), и блокирует действия пользователя (Рисунок 19).

```
DELETE FROM app.audio_archives WHERE archive_id = 1;
DELETE FROM app.audio_archives WHERE archive_id = 2;
DELETE FROM app.audio_archives WHERE archive_id = 3;
DELETE FROM app.audio_archives WHERE archive_id = 4;
DELETE FROM app.audio_archives WHERE archive_id = 5;
DELETE FROM app.audio_archives WHERE archive_id = 6;
```

Data Output | Messages | Notifications

```
ERROR:  Подозрительная активность: слишком много удалений из app.audio_archives за короткий период (пользователь: postgres)
CONTEXT:  PL/pgSQL function app.tg_audio_archives_detect_mass_delete() line 13 at RAISE

SQL state: P0001
```

Рисунок 19 – Проверка третьего триггера

Четвертый триггер для автоматического заполнения даты и времени в таблицах при добавлении новой записи (Рисунок 20).

```
INSERT INTO app.projects (studio_id, title, original_language, description, project_code, created_at)
VALUES (1, 'Test Auto Date', 'ru', 'проверка автодаты', 'PRJ-AUTO-002', NULL);

SELECT * FROM app.projects WHERE project_code = 'PRJ-AUTO-002';
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page No: 1 of 1

	project_id [PK] integer	studio_id integer	title character	original charact	descrip text	client_c text	budget numeric	start_da date	end_date date	status character varying (20)	created_at timestamp with time zone
1	16	1	Test A...	ru	про...	[null]	[null]	[null]	[null]	active	2026-05-05 10:06:47.488569+...

Рисунок 20 – Проверка четвертого триггера

Пятый триггер будет блокировать смену паролей пользователей БД без специального разрешения и возвращать сообщение об ошибке (Рисунок 21).

```
UPDATE app.actors
SET password_hash = md5('pass2')
WHERE actor_id = 1;
```

Data Output | Messages | Notifications

```
ERROR:  Смена пароля запрещена для пользователя "postgres": нет специального разрешения
CONTEXT:  PL/pgSQL function app.tg_actors_block_password_change() line 12 at RAISE

SQL state: P0001
```

Рисунок 21 – Проверка пятого триггера

3. Выводы о применении индексов и триггеров в БД

На основе полученных данных о производительности запросов с помощью EXPLAIN ANALYZE к таблице app.index_benchmark мы сможем сравнить разницу до и после индексации.

До применения индекса B-TREE поиск по электронной почте с использованием условия равенства приводит к полному последовательному сканированию всей таблицы и требует в среднем 11 миллисекунд. После создания B-TREE индекса по столбцу customer_email время выполнения этого же запроса сократилось до 0,04 миллисекунды, что означает ускорение более чем в 275 раз.

Запрос к столбцу order_code (код заказа), выполненный без индекса, занимал около 9,8 миллисекунды. После создания HASH индекса время снизилось до 0,02 миллисекунды, увеличение производительности примерно в 490 раз.

Пространственный запрос, отбирающий точки, попадающие в заданный прямоугольник, без индекса обрабатывался 13,7 миллисекунды, а с GiST-индексом 0,8 миллисекунды, то есть ускорился в 17 раз.

Запрос на поиск двадцати ближайших к началу координат точек с оператором расстояния без индекса выполнялась 25,2 миллисекунды; применение SP-GiST индекса сократило это время до 0,3 миллисекунды, ускорение в 86 раза.

Для запроса, подсчитывающего количество записей за последние 30 минут, время выполнения до индекса было 19,3, а после применения BRIN индекса стало 0,01.

Поиск записей, содержащих в массиве тегов значение «project_42», до GIN-индекса требовал 21,3 миллисекунды, а после 2,9 миллисекунды.

При этом важно, что каждому запросу нужен свой подходящий индекс. B-Tree для поиска по точному совпадению или диапазону, Hash только для точного равенства, GiST для проверки вхождения гео-точки в прямоугольник,

SP-GiST для быстрой сортировки по расстоянию от точки, BRIN для запросов по диапазону дат, когда данные физически отсортированы по времени, GIN для поиска в массивах или тексте, когда нужно найти строки, содержащие определённый элемент.

Применённые триггеры обеспечивают целостность и безопасность данных.

Триггер контроля уникальности кода предотвращает вставку дубликатов и формирует сообщение об ошибке, что исключает появление двух объектов с одинаковым идентификатором.

Триггер автоматического заполнения даты создания не позволяет оставить поле пустым.

Триггер синхронизации зарплатной ставки переносит актуальное значение из контракта в каждую связанную сессию записи сразу после изменения контракта.

Триггер обнаружения подозрительной активности ведёт счётчик удалений из таблицы аудиоархивов за последнюю минуту и при превышении порога блокирует дальнейшие удаления, выдавая ошибку.

Триггер блокировки смены пароля запрещает изменять хеш пароля актёра, если текущий пользователь базы данных не внесён в разрешительный список password_change_acl.

Вывод

В ходе выполнения лабораторной работы для базы данных студий дубляжа были решены две задачи: оптимизация запросов с помощью разных типов индексов и автоматизация поддержания целостности данных с помощью триггеров.

До индексации все запросы выполнялись последовательным сканированием и требовали несколько миллисекунд. После создания соответствующих индексов время сократилось несколько раз: B-Tree ускорил поиск по email в 275 раз, Hash для кода заказа в 490 раз, GiST для пространственного поиска в 17 раз, SP-GiST для сортировки по расстоянию в 86 раз, BRIN для выборки по дате за последние 30 минут более чем в 1000 раз, GIN для поиска по тегу массива примерно в 7 раз.

Реализованные триггеры обеспечивают автоматический контроль целостности и безопасности данных на уровне СУБД. Триггер tg_projects_check_unique_code предотвращает вставку дублирующихся кодов проектов, формируя сообщение об ошибке. Триггер tg_projects_fill_created_at проставляет текущую временную метку, если поле created_at не заполнено явно. Триггер tg_contracts_sync_salary_to_sessions при изменении зарплаты в контракте мгновенно обновляет ставки во всех связанных сессиях записи. Триггер tg_audio_archives_detect_mass_delete блокирует удаление, если пользователь за одну минуту пытается удалить несколько аудиоархивов, тем самым защищая от подозрительной активности. Триггер tg_actors_block_password_change запрещает изменение пароля актёра, если текущий пользователь базы данных не имеет разрешения в таблице password_change_acl.

Индексы позволяют повысить скорость выполнения запросов к большим таблицам, а триггеры нужны для автоматического поддержания целостности и предотвращения некорректных операций с данными.

Приложение А

```
SET search_path TO app, public;

CREATE TABLE app.index_benchmark (
    id BIGSERIAL PRIMARY KEY,
    order_code BIGINT NOT NULL,
    customer_email TEXT NOT NULL,
    amount NUMERIC(12,2) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    geolocation POINT NOT NULL,
    description TEXT NOT NULL,
    tags TEXT[] NOT NULL
);

CREATE OR REPLACE FUNCTION app.fill_index_benchmark(p_rows
INTEGER DEFAULT 300000)
RETURNS VOID
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO app.index_benchmark (order_code, customer_email, amount,
created_at, geolocation, description, tags)
    SELECT
        1000000 + g AS order_code,
        'client_' || (g % 50000) || '@mail.local' AS customer_email,
        ROUND((10 + random() * 10000)::numeric, 2) AS amount,
        NOW() - ((p_rows - g) || ' minutes')::interval AS created_at,
        point((random() * 2000 - 1000)::float8, (random() * 2000 - 1000)::float8) AS
geolocation,
```

```
        'Order #' || g || ' for project ' || (g % 900) || ' with payload ' ||
encode(digest(g::text, 'md5'), 'hex') AS description,
        ARRAY[
            CASE WHEN g % 2 = 0 THEN 'music' ELSE 'voice' END,
            CASE WHEN g % 3 = 0 THEN 'urgent' ELSE 'normal' END,
            'project_' || (g % 100)::text
        ]::text[] AS tags
    FROM generate_series(1, p_rows) AS s(g);
END;
$$;

-- До индексов
\echo 'До индексов'
\echo 'B-Tree'
EXPLAIN ANALYZE
SELECT *
FROM app.index_benchmark
WHERE customer_email = 'client_12345@mail.local';

\echo 'Hash'
EXPLAIN ANALYZE
SELECT *
FROM app.index_benchmark
WHERE order_code = 1187654;

\echo 'GiST'
EXPLAIN ANALYZE
SELECT COUNT(*)
FROM app.index_benchmark
WHERE geolocation <@ box(point(-50, -50), point(50, 50));
```

```
\echo 'SP-GiST'
EXPLAIN ANALYZE
SELECT id, geolocation
FROM app.index_benchmark
ORDER BY geolocation <-> point(0, 0)

\echo 'BRIN'
EXPLAIN ANALYZE
SELECT COUNT(*)
FROM app.index_benchmark
WHERE created_at >= NOW() - INTERVAL '30 minutes';

\echo 'GIN'
EXPLAIN ANALYZE
SELECT COUNT(*)
FROM app.index_benchmark
WHERE tags @> ARRAY['project_42']::text[];

CREATE INDEX IF NOT EXISTS idx_bench_btree_email ON
app.index_benchmark USING btree (customer_email);
CREATE INDEX IF NOT EXISTS idx_bench_hash_order_code ON
app.index_benchmark USING hash (order_code);
CREATE INDEX IF NOT EXISTS idx_bench_gist_geo ON app.index_benchmark
USING gist (geolocation);
CREATE INDEX IF NOT EXISTS idx_bench_spgist_geo ON
app.index_benchmark USING spgist (geolocation);
CREATE INDEX IF NOT EXISTS idx_bench_brin_created_at ON
app.index_benchmark USING brin (created_at);
CREATE INDEX IF NOT EXISTS idx_bench_gin_tags ON app.index_benchmark
USING gin (tags);
```

```
-- После индексов
\echo 'После индексов'
\echo 'B-Tree'
EXPLAIN ANALYZE
SELECT *
FROM app.index_benchmark
WHERE customer_email = 'client_12345@mail.local';

\echo 'Hash'
EXPLAIN ANALYZE
SELECT *
FROM app.index_benchmark
WHERE order_code = 1187654;

\echo 'GiST'
EXPLAIN ANALYZE
SELECT COUNT(*)
FROM app.index_benchmark
WHERE geolocation <@ box(point(-50, -50), point(50, 50));

\echo 'SP-GiST'
EXPLAIN ANALYZE
SELECT id, geolocation
FROM app.index_benchmark
ORDER BY geolocation <-> point(0, 0)
LIMIT 20;
```

```
\echo 'BRIN'
EXPLAIN ANALYZE
SELECT COUNT(*)
FROM app.index_benchmark
WHERE created_at >= NOW() - INTERVAL '30 minutes';

\echo 'GIN'
EXPLAIN ANALYZE
SELECT COUNT(*)
FROM app.index_benchmark
WHERE tags @> ARRAY['project_42']::text[];
```

Приложение Б

```
SET search_path TO app, audit, public;

ALTER TABLE app.projects
   ADD COLUMN IF NOT EXISTS project_code VARCHAR(50);

ALTER TABLE app.recording_sessions
   ADD COLUMN IF NOT EXISTS contract_salary_snapshot NUMERIC(10,2);

ALTER TABLE app.actors
   ADD COLUMN IF NOT EXISTS login_name VARCHAR(100),
   ADD COLUMN IF NOT EXISTS password_hash TEXT;

CREATE TABLE IF NOT EXISTS app.password_change_acl (
   db_login VARCHAR(100) PRIMARY KEY,
   can_change_password BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE IF NOT EXISTS audit.delete_watch_log (
   event_id BIGSERIAL PRIMARY KEY,
   db_user VARCHAR(100) NOT NULL,
   table_name VARCHAR(100) NOT NULL,
   deleted_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Триггер проверки уникальности значения перед вставкой
CREATE OR REPLACE FUNCTION app.tg_projects_check_unique_code()
RETURNS TRIGGER
LANGUAGE plpgsql
```

```
AS $$
BEGIN
   IF NEW.project_code IS NOT NULL AND EXISTS (
      SELECT 1
      FROM app.projects p
      WHERE p.project_code = NEW.project_code
   ) THEN
      RAISE EXCEPTION 'Проект с кодом "%" уже существует',
NEW.project_code;
   END IF;
   RETURN NEW;
END;
$$;

CREATE TRIGGER trg_projects_check_unique_code
BEFORE INSERT ON app.projects
FOR EACH ROW
EXECUTE FUNCTION app.tg_projects_check_unique_code();

-- Триггер автозаполнения даты при вставке
CREATE OR REPLACE FUNCTION app.tg_projects_fill_created_at()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
   IF NEW.created_at IS NULL THEN
      NEW.created_at := NOW();
   END IF;
   RETURN NEW;
END;
```

```
$$;

CREATE TRIGGER trg_projects_fill_created_at
BEFORE INSERT ON app.projects
FOR EACH ROW
EXECUTE FUNCTION app.tg_projects_fill_created_at();

-- Триггер обновления поля в связанной таблице
CREATE OR REPLACE FUNCTION app.tg_contracts_sync_salary_to_sessions()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
   UPDATE app.recording_sessions rs
      SET contract_salary_snapshot = NEW.salary_amount
    WHERE rs.contract_id = NEW.contract_id;
   RETURN NEW;
END;
$$;

CREATE TRIGGER trg_contracts_sync_salary_to_sessions
AFTER UPDATE OF salary_amount ON app.contracts
FOR EACH ROW
WHEN (OLD.salary_amount IS DISTINCT FROM NEW.salary_amount)
EXECUTE FUNCTION app.tg_contracts_sync_salary_to_sessions();

-- Триггер обнаружения массовых DELETE и блокировка действий
CREATE OR REPLACE FUNCTION
app.tg_audio_archives_detect_mass_delete()
RETURNS TRIGGER
```

```
LANGUAGE plpgsql
AS $$
DECLARE
   recent_count INTEGER;
BEGIN
   SELECT COUNT(*)
     INTO recent_count
     FROM audit.delete_watch_log l
    WHERE l.db_user = CURRENT_USER
      AND l.table_name = 'app.audio_archives'
      AND l.deleted_at > NOW() - INTERVAL '1 minute';

   IF recent_count >= 5 THEN
       RAISE EXCEPTION 'Подозрительная активность: слишком много
удалений из app.audio_archives за короткий период (пользователь: %)',
CURRENT_USER;
   END IF;

   INSERT INTO audit.delete_watch_log(db_user, table_name)
   VALUES (CURRENT_USER, 'app.audio_archives');

   RETURN OLD;
END;
$$;

CREATE TRIGGER trg_audio_archives_detect_mass_delete
BEFORE DELETE ON app.audio_archives
FOR EACH ROW
EXECUTE FUNCTION app.tg_audio_archives_detect_mass_delete();
```

```
-- Триггер блокировки смены пароля без разрешения
CREATE OR REPLACE FUNCTION app.tg_actors_block_password_change()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
   is_allowed BOOLEAN;
BEGIN
   IF NEW.password_hash IS DISTINCT FROM OLD.password_hash THEN
      SELECT a.can_change_password
        INTO is_allowed
        FROM app.password_change_acl a
       WHERE a.db_login = CURRENT_USER;

      IF COALESCE(is_allowed, FALSE) = FALSE THEN
         RAISE EXCEPTION 'Смена пароля запрещена для пользователя "%": нет специального разрешения', CURRENT_USER;
      END IF;
   END IF;

   RETURN NEW;
END;
$$;

CREATE TRIGGER trg_actors_block_password_change
BEFORE UPDATE OF password_hash ON app.actors
FOR EACH ROW
EXECUTE FUNCTION app.tg_actors_block_password_change();
```

```
-- Демонстрация работы триггеров
-- Проверка уникальности при вставке
INSERT INTO app.projects (studio_id, title, original_language, description,
project_code, created_at)
VALUES (1, 'DEMO Trigger Project', 'ru', 'demo row', 'PRJ-DEMO-001', NULL);

-- Обновление связанных данных
UPDATE app.contracts
SET salary_amount = salary_amount + 100
WHERE contract_id = 1;

SELECT session_id, contract_id, contract_salary_snapshot
FROM app.recording_sessions
WHERE contract_id = 1
ORDER BY session_id

-- Блокировка массовых удалений
DELETE FROM app.audio_archives WHERE archive_id = 1;
DELETE FROM app.audio_archives WHERE archive_id = 2;
DELETE FROM app.audio_archives WHERE archive_id = 3;
DELETE FROM app.audio_archives WHERE archive_id = 4;
DELETE FROM app.audio_archives WHERE archive_id = 5;
DELETE FROM app.audio_archives WHERE archive_id = 6;

-- Блокировка смены пароля
UPDATE app.actors
SET password_hash = md5('pass2')
WHERE actor_id = 1;
```

```
-- Проверка автодаты
INSERT INTO app.projects (studio_id, title, original_language, description,
project_code, created_at)
VALUES (1, 'Test Auto Date', 'ru', 'проверка автодаты', 'PRJ-AUTO-002',
NULL);

SELECT * FROM app.projects WHERE project_code = 'PRJ-AUTO-002';
```